

8.1 Introduction (Simple Version)

1. What this chapter is about:

- In simulations, we often need **random numbers** to mimic real-life randomness (like customer arrivals, demand sizes, or service times).
- Generating a random number that fits a certain **probability distribution** (like exponential, Poisson, or gamma) is called “**generating a random variate.**”
- Before generating these random numbers, we **assume the distribution and its parameters are already known.**Example:
- In a queue (line) simulation, we need random **arrival times** and **service times**.
- In an inventory simulation, we need random **demand sizes**.

2. The main tool needed:

- Every method of generating random variates starts from a source of **uniform random numbers between 0 and 1** (written as **U(0,1)**).
- A **good, reliable random-number generator** is essential.
- If the generator is poor, we cannot generate random numbers correctly.
- Most modern computers and simulation software have **good random-number generators**, but older ones might be unreliable.

3. Choosing an algorithm to generate random numbers:

There are **different algorithms** to generate random variates from a distribution. Choosing one involves **trade-offs**:

Factors to consider:

a) Exactness:

- Prefer algorithms that generate numbers **exactly following the desired distribution**.
- Older approximate methods (like summing 12 U(0,1) numbers to get a “normal” number) exist, but exact methods are now available for most distributions.

4. b) Efficiency:

- Two things matter:
 1. **Storage space:** Some algorithms need large tables or constants.
 2. **Execution time:**
 - **Marginal execution time:** Time to generate **one random number**.
 - **Setup time:** Time to compute constants or tables for the algorithm.
 3. Usually, marginal execution time matters more because we generate **lots of numbers**, unless the distribution **changes often**.

5. c) Complexity:

- More efficient algorithms can be **complicated**.
- Ask yourself: Is the extra efficiency worth the effort to **understand and implement** it?
- For permanent software, complexity may be okay. For **one-time simulations**, simpler methods are better.

6. d) Technical issues:

- Some algorithms require random numbers from **other distributions** — not ideal.
- Some algorithms work well for some **parameter values** but not for others. We prefer **robust algorithms** that work well for all values.
- For **variance reduction techniques** (methods to reduce randomness in simulations), some algorithms make it easier to synchronize random numbers across simulations.
- Example: The **inverse-transform method** helps in synchronizing numbers and reducing variance.

4. References and software:

- There are many books and papers on random-number generation.
- Some software packages already include functions to generate random numbers for **many distributions**.

5. Structure of the chapter:

- **Section 8.2:** General approaches to generating random numbers.
- **Sections 8.3 & 8.4:** Algorithms for continuous and discrete distributions.
- **Sections 8.5 & 8.6:** Specialized topics like generating **correlated random numbers** and **arrival processes**.

8.2.1 Inverse Transform Method (Simple Explanation)

What is the inverse-transform method?

- It's a **general way to generate random numbers** (random variates) from any probability distribution. Works for both continuous and discrete distributions.

Algorithm for continuous distributions

Intuition:

- The **density function** ($f(x)$) is like the “chance” of (X) being near (x).
- Steep parts of ($F(x)$) correspond to high ($f(x)$) $\rightarrow$ more (X) values will fall there.

- Flat parts of $(F(x))$ correspond to low $(f(x)) \rightarrow$ fewer (X) values fall there.
- So, the inverse-transform method **maps uniform numbers into the shape of the desired distribution.**

Algorithm for discrete distributions

Suppose (X) is discrete with **probabilities** $(p(x_i) = P(X = x_i))$. Steps:

1. Generate $(U \sim U(0,1))$.
2. Find the **smallest** (i) such that $(U \leq F(x_i))$, where $(F(x_i) = \sum_{j \leq i} p(x_j))$.
3. Set $(X = x_i)$.

Intuition:

- Divide the interval $[0,1]$ into subintervals based on probabilities.
- Random number (U) lands in a subinterval $\rightarrow$ pick the corresponding (X) .

4 General formula (works for all types)

$$X = \min \{x : F(x) \geq U\}$$

- This works for **continuous, discrete, or mixed distributions** (even distributions with jumps or flat parts).

5 Advantages of the inverse-transform method

1. **Simple to understand and implement.**
2. **Works for continuous, discrete, and mixed distributions.**
3. **Helps variance reduction techniques** (like common random numbers and antithetic variates)
 - Using the same (U) can make two outputs positively correlated.
 - Using $(1-U)$ can make them negatively correlated.
4. **Easy to generate truncated distributions**
 - Example: Only want (X) between (a) and (b) :

$$V = F(a) + [F(b) - F(a)] U, \quad X = F^{-1}(V)$$
5. **Useful for order statistics** (like min/max or the i th smallest value in a sample)

6 Disadvantages

1. **Sometimes hard to compute $(F^{-1})(U)$**
 - Some distributions (like normal or gamma) don't have a simple inverse.

- Can use **numerical methods**, approximations, or special libraries.
2. **May not be the fastest**
- For some distributions, other algorithms are quicker.

Exam tips

- **Inverse-transform method = Uniform $\rightarrow$ Desired distribution**
- **Continuous:** $(X = F^{-1}(U))$
- **Discrete:** (X) is the smallest value such that $(F(x_i) \geq U)$
- **Intuition:** Uniform numbers get “stretched” into the shape of the desired distribution.
- Useful for **variance reduction, truncation, and order statistics**.

8.2.2 Composition Method (Simple Explanation)

What is the composition method?

- Used when a distribution (F) can be written as a **weighted combination of simpler distributions** $(F_1, F_2, \dots)$. Idea: It's easier to **sample from (F_j)** than from (F) directly.

Mathematically:

Here, (p_j) = probability of picking distribution (F_j) . Continuous and discrete cases both work.

Algorithm (General Steps)

1. **Choose a distribution:** Generate a random integer (J) such that $P(J = j) = p_j$
 - This selects which (F_j) you will sample from.
2. **Generate X** from the chosen (F_J) independently.

3 Example 8.3 – Double-Exponential (Laplace) Distribution – SIR

4 Example 8.4 – Right-Trapezoidal Distribution

5 Key Points / Exam Tips

- **Composition = Weighted combination of simpler distributions**
- Steps:
 1. Pick which component (F_j) to sample from (weighted by (p_j))
 2. Generate X from that component independently

- **Requires at least 2 random numbers** (one for selecting the component, one for sampling)
- Can combine **composition + inverse-transform** to simplify sampling.
- Useful for:
 1. Distributions that are “mixtures” (double-exponential, trapezoidal, gamma, beta)
 2. Faster sampling than direct inverse-transform in some cases

8.2.3 Convolution Method

1 What is the convolution method?

- Used when the **random variable (X)** we want can be expressed as a **sum of simpler IID random variables**:

$$X = Y_1 + Y_2 + \dots + Y_m$$

- Here, each (Y_j) is **easy to generate**.
- This is different from the composition method:
 - **Convolution:** $(X) = \text{sum of variables}$
 - **Composition:** Distribution $(F_X) = \text{weighted sum of other distributions}$

Algorithm

1. Generate $(Y_1, Y_2, \dots, Y_m)$ independently, each with distribution (G) .
2. Return $(X = Y_1 + Y_2 + \dots + Y_m)$.

Valid because (X) and the sum of the (Y_j) 's have the same distribution.

Example 8.5 – m-Erlang Distribution

- m-Erlang random variable (X) with mean (b) :

$$X = Y_1 + Y_2 + \dots + Y_m$$
- Each $(Y_j) = \text{exponential random variable with mean } (b/m)$.
- Steps to generate X :
 1. Generate (m) independent exponential variables $(Y_1, \dots, Y_m)$
 2. Sum them: $(X = Y_1 + \dots + Y_m)$

Note: If (m) is large, this can be slow. Better methods exist using the **gamma distribution**.

Key Points

- Convolution = sum of IID intermediate variables
- Simple to implement if (Y_j) 's are easy to generate
- Efficiency can be an issue for large (m)

8.2.4 Acceptance-Rejection Method

Intuition:

- Think of plotting $(f(x))$ under $(t(x))$.
- Random points under $(t(x))$ are “thinned out”:
 - Keep points where $(f(x))$ is high
 - Reject points where $(f(x))$ is low

Example 8.6 – Beta(4,3) Distribution

Improved Acceptance-Rejection – Piecewise Majorizing Function (Example 8.7)

- Instead of constant (t) , use **piecewise-linear $t(x)$** fitting closely above $(f(x))$
- Acceptance probability rises (e.g., 0.78 instead of 0.48)
- Tradeoff: Generating (Y) from $(r(x))$ becomes slightly more complex, often solved by **composition + inverse-transform**

Key Points / Exam Tips

- **Acceptance-rejection = indirect method**
- Choose $(t(x))$ to balance:
 1. Easy generation of (Y)
 2. High probability of acceptance (small (c))
- Works for **any distribution** and can extend to **multi-dimensional spaces**
- Often combined with **other methods** (inverse-transform, composition) for efficiency

Method	Idea	Steps	Example
Convolution	$X = \text{sum of IID } Y_j$	Generate $Y_j \rightarrow \text{sum}$	m-Erlang distribution
Acceptance-Rejection	Generate candidates $\rightarrow$ accept/reject	Generate $Y \sim r$, $U \sim U(0,1)$, accept if $U \leq f(Y)/t(Y)$	Beta(4,3), t constant or piecewise

8.3 Generating Continuous Random Variates

- Goal: Generate **continuous random variables** (like uniform, exponential, etc.) from a computer's uniform random number generator.
- Focus: Present **one simple, exact method** per distribution. Other faster or more complex methods exist (see references).
- Approach: Use **inverse-transform method**, **composition**, **convolution**, or **acceptance-rejection** where appropriate.

8.3.1 Uniform Distribution (U(a, b))

8.3.2 Exponential Distribution

Distribution	Method	Formula/Steps	Notes
Uniform (U(a,b))	Inverse-transform	$(X = a + (b-a)U)$	Precompute (b-a) for speed
Exponential (Exp(b))	Inverse-transform	$(X = -b \ln U)$	Most of the time spent on $(\ln)$ evaluation

This section basically sets up **exact, simple methods for commonly used continuous distributions** before moving on to more complicated ones (like normal, gamma, beta, etc.) in later subsections.

Random number generator (RNG):

Generates basic uniform random numbers (usually between 0 and 1).

Random variate generator (RVG):

Uses those uniform random numbers to generate values from a specific distribution (like normal, exponential, etc.).

Full form of CDF: Cumulative Distribution Function

8.3.6 Normal Distribution (Generating (N(m, s²)))

1 Box-Muller Method (1958)

2 Polar (Marsaglia) Method (1964)

3 Inverse-Transform Method

- Not practical directly for normal distribution:
 - $(F^{-1}(u))$ for normal has **no closed-form**, requires numerical methods.
- Example implementation: Moro (1995) or IMSL library routines.

4 Other Notes

- **Applications:**
 - Generating normal random numbers for **acceptance-rejection methods** of other distributions (gamma, beta, lognormal).
 - Monte Carlo simulations requiring null distributions or antithetic variates.
- **Faster algorithms:** Kinderman & Ramage (1976) → 30% faster than polar method.

Method	Key Idea	Output	Pros / Cons
Box-Muller	Trig transform of $U(0,1)$	2 IID $N(0,1)$	Simple, but slow; potential dependence if using single LCG stream
Polar / Marsaglia	Rejection + scaling, avoids trig	2 IID $N(0,1)$	Faster; uses random number of uniforms; widely used
Inverse-transform	Solve $(F^{-1})(U)$ numerically	1 $N(0,1)$	Exact, but slower; needs numerical inversion
Kinderman-Ramage	Advanced algorithm	1 $N(0,1)$	Very fast; more complex

Standard Deviation:

Standard deviation is a measure of how much the values in a dataset spread out from the mean (average).

In simple words:

It tells us how far the data values are, on average, from the mean.



If standard deviation is **small** → data are close to the mean.



If standard deviation is **large** → data are widely spread out.